

DATABASE MANAGEMENT SYSTEMS

B.Tech IV Semester



Prepared
By
Dr V.SITHARAMULU

GITAM (Deemed to be University)

GITAM, Rudraram, Patancheru mandal, Hyderabad - 502329 Telangana, India

MODULE-I : INTRODUCTION TO DBMS AND DATABASE DESIGN

Introduction to DBMS: File system vs DBMS, advantages of DBMS, storage data, queries, DBMS structure, Types of Databases– Hierarchical, Network, Relational, Key-Value, Object-oriented, XML DB.

Overview of File Structures in database.

E-R model: Entities, attributes and entity sets, relationship and relationship set, mapping cardinalities, keys, features of ER model, conceptual database design with ER model.

MODULE-II : RELATIONAL MODEL AND BASIC SQL

Relational model: Integrity constraints over relations and enforcement, querying relational data, logical database design, views, destroying/altering tables and views.

Basic SQL: Introduction to SQL, Basic SQL Queries: DML, DDL, DCL, TCL

MODULE-III ADVANCES SQL AND PL/SQL

Structured Query Language (SQL): Select Commands, Union, Intersection, Except, Nested Queries, Aggregate Operators, Null values, Relational set operators, SQL join operators

Relational Algebra(RA): Selection, Projection, Set operations, Joins

Relational Calculus (TRC, DRC): Tuple Relational Calculus, Domain Relational Calculus

PL/SQL, Assertions, Triggers

MODULE-IV : SCHEMA REFINEMENT AND NORMAL FORMS

Introduction to Schema Refinement, Functional Dependencies, Reasoning about Functional Dependencies. Normal Forms, Properties of Decomposition, Normalization, different types of dependencies.

MODULE-V INTRODUCTION TO TRANSACTION MANAGEMENT, CONCURRENCY CONTROL AND CRASH RECOVERY

Introduction to Transaction Management: ACID properties, Transactions and Schedules, Concurrent Execution of Transactions, Lock-Based Concurrency Control.

Concurrency Control: 2PL, Serializability and Recoverability, Introduction to Lock Management, Lock Conversions, Dealing with Deadlocks, Concurrency control without locking.

Crash Recovery: Aries, Recovering from a System Crash

TEXT BOOKS:

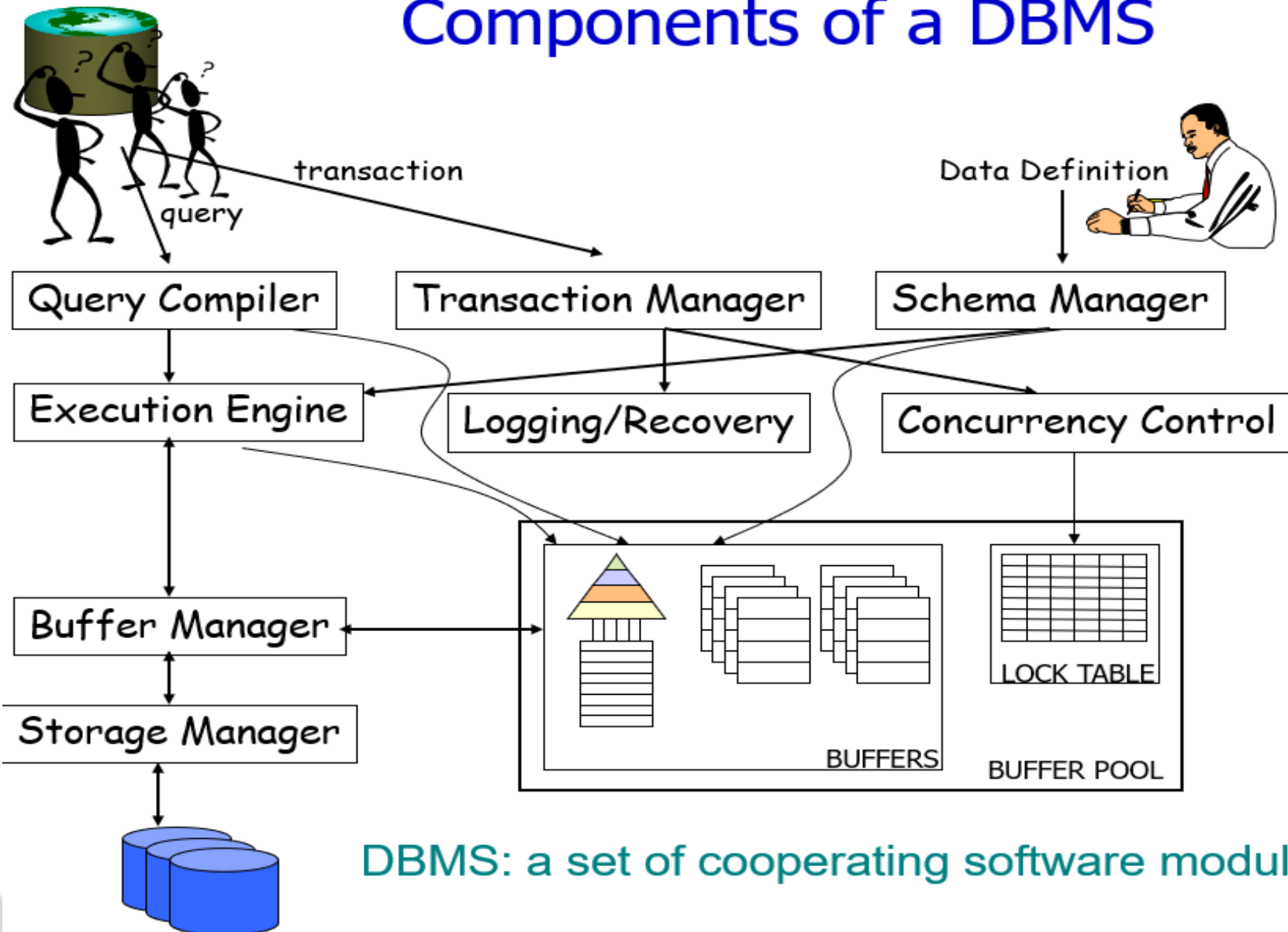
1. Raghu Ramakrishnan and Johannes Gehrke, Database Management Systems, 3rd Edition, McGraw-Hill, 2014, 978-8131769591.
2. H.F.Korth and A.silberschatz, Database System Concepts, 7th Edition, McGraw-Hill, 2021 , 978-9390727506.

REFERENCE BOOKS:

1. D. Ullman,, Principles of Database and Knowledge – Base Systems, Vol 1, 1st Edition, Computer Science Press, 1990
2. Ramez Elmasri, Shamkant B. Navathe, Fundamentals of Database Systems, Pearson Education, 2016

Transaction Management

Components of a DBMS



Transaction Management

Introduction to Transaction Management

Transaction Management is a fundamental concept in **Database Management Systems (DBMS)** that ensures **data consistency, reliability, and integrity** when multiple users access or modify the database simultaneously.

What is a Transaction?

A transaction is a sequence of one or more database operations (like INSERT, UPDATE, DELETE) treated as a single logical unit of work.

Example:

Transferring money from Account A to Account B:

```
UPDATE Account SET balance = balance - 1000 WHERE acc_no = 'A';
```

```
UPDATE Account SET balance = balance + 1000 WHERE acc_no = 'B';
```

Both operations together form **one transaction**.

- If both succeed → **Commit**
- If any fails → **Rollback**

Transaction Management

E.g. transaction to transfer \$50 from account A to account B:

1. **read**(A)
2. $A := A - 50$
3. **write**(A)
4. **read**(B)
5. $B := B + 50$
6. **write**(B)

Two main issues to deal with:

Failures of various kinds, such as hardware failures and system crashes
Concurrent execution of multiple transactions

Very valuable properties of DBMSs

without these, DBMSs would be much less useful

Based on concept of transactions with ACID properties

Transaction Management

E.g. transaction to transfer \$50 from account A to account B:

1. **read**(A)
2. $A := A - 50$
3. **write**(A)
4. **read**(B)
5. $B := B + 50$
6. **write**(B)

Two main issues to deal with:

Failures of various kinds, such as hardware failures and system crashes

Concurrent execution of multiple transactions

Concurrent Execution

- Both transactions are executed **simultaneously (interleaved)**.

Problem: Lost Update

- This improves:

- **Throughput** (more work done)
- **Response time**
- **CPU utilization**

- **Example:**

	T1:	T2:
t0:	tmp1 := read(X)	
t1:		tmp2 := read(X)
t2:	tmp1 := tmp1 - 20	
t3:		tmp2 := tmp2 + 10
t4:	write tmp1 into X	
t5:		write tmp2 into X

ACID Properties

Every transaction must satisfy the **ACID properties**:

A – Atomicity (All or Nothing)

- A transaction is either fully completed or not executed at all.
- If one operation fails → entire transaction is rolled back.

C – Consistency

- Database moves from one valid state to another valid state.
- Constraints (like keys, rules) must not be violated.

If the database is initially consistent and every transaction preserves consistency, then after execution the database remains consistent.

I – Isolation

- Transactions execute independently without interference.
- Intermediate states are hidden from other transactions.

D – Durability

- Once committed, changes are permanent even if system crashes.

ACID Properties

A – Atomicity Example: Bank Money Transfer

Consider a transaction where ₹1000 is transferred from Account A to Account B.

Transaction Steps:

1. Read balance of A
2. $A = A - 1000$
3. Write updated balance of A

4. Read balance of B
5. $B = B + 1000$
6. Write updated balance of B

Case 1: Successful Execution

- All steps execute properly
- Final result:
 - A balance decreases by 1000
 - B balance increases by 1000

→ **Transaction COMMIT**

Case 2: Failure in Between

Suppose:

- Steps 1–3 are completed (money deducted from A)
- System crashes before step 6 (money not added to B)

Without Atomicity:

- A loses ₹1000
- B does not receive ₹1000

→ **Data inconsistency (money lost!)**

With Atomicity:

- System detects failure
 - Entire transaction is **ROLLED BACK**
 - Deduction from A is undone
 - Database returns to original state
- No money is lost, consistency maintained

ACID Properties

C – Consistency

Example 1: Bank Balance Constraint

- Assume a rule:
Account balance should never be negative

Initial State (Valid):

- Account A balance = ₹5000

Transaction: Withdraw ₹6000

```
UPDATE Account  
SET balance = balance - 6000  
WHERE acc_no = 'A';
```

Without Consistency:

- New balance = ₹-1000 ✗ (Invalid state)
- Constraint violated

With Consistency:

- DBMS checks constraint
- Transaction is **rejected/rolled back**
- Balance remains ₹5000 ✓

Example 2: Foreign Key Constraint Tables:

- Student(StudentID)
- Enrollment(StudentID)

Rule:

- Enrollment.StudentID must exist in Student table

Transaction:

```
INSERT INTO Enrollment VALUES (101);
```

Case:

- StudentID 101 does NOT exist in Student table

Result:

- Transaction is **rejected**
- Database remains consistent

ACID Properties

I – Isolation

Definition:

Isolation ensures that **multiple transactions execute independently**, without interfering with each other.

The result should be the same as if transactions were executed **one after another (serially)**.

Assume initial value:

X = 100

Transaction T1

Read(X)

$X = X - 20$

Write(X)

Transaction T2

Read(X)

$X = X + 10$

Write(X)

Without Isolation (Concurrent Execution)

1. T1 reads X = 100

2. T2 reads X = 100

3. T1 writes X = 80

4. T2 writes X = 110

- Final value = **110 (WRONG)**
- Correct value should be **90**

With Isolation (Serial Execution)

Case 1: T1 → T2

• $X = 100 \rightarrow 80 \rightarrow 90 \checkmark$

Case 2: T2 → T1

• $X = 100 \rightarrow 110 \rightarrow 90 \checkmark$

- Final value = **90 (CORRECT)**

Problems Prevented by Isolation

- **Dirty Read** – Reading uncommitted data
- **Lost Update** – One update overwrites another
- **Unrepeatable Read** – Same query gives different results

Transaction Management

D – Durability

Definition:

Durability ensures that once a transaction is **committed**, its changes are **permanently stored** in the database, even if there is a **system crash, power failure, or error**.

- “Committed data will never be lost.”

Example: Bank Deposit

Suppose a user deposits ₹2000 into an account.

Transaction:

```
UPDATE Account
SET balance = balance + 2000
WHERE acc_no = 'A';
COMMIT;
```

Case 1: After Commit

- Balance is updated successfully
- Transaction is committed

Even if the system crashes immediately after,
Updated balance remains (safe)

Case 2: Without Durability

- System crashes after commit
- Changes are lost ✗
This should **NOT happen** in a DBMS

How Durability is Achieved

DBMS uses:

- Log files (Write-Ahead Logging - WAL)
- Checkpointing
- Backup & Recovery mechanisms
- Stable storage (disk/SSD)

Transaction State

Active – the initial state; the transaction stays in this state while it is executing

Partially committed – after the final statement has been executed.

Failed -- after the discovery that normal execution can no longer proceed.

Aborted – after the transaction has been rolled back and the database restored to its state prior to the start of the transaction.

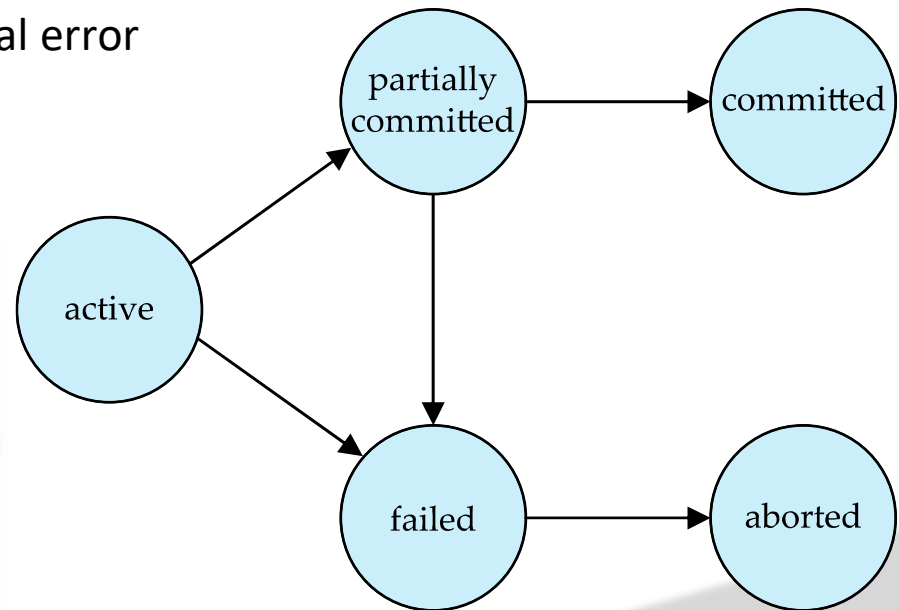
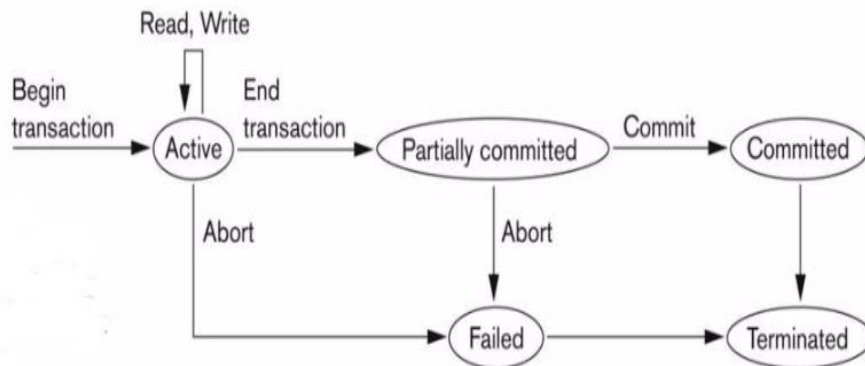
Two options after it has been aborted:

Restart the transaction

Can be done only if no internal logical error

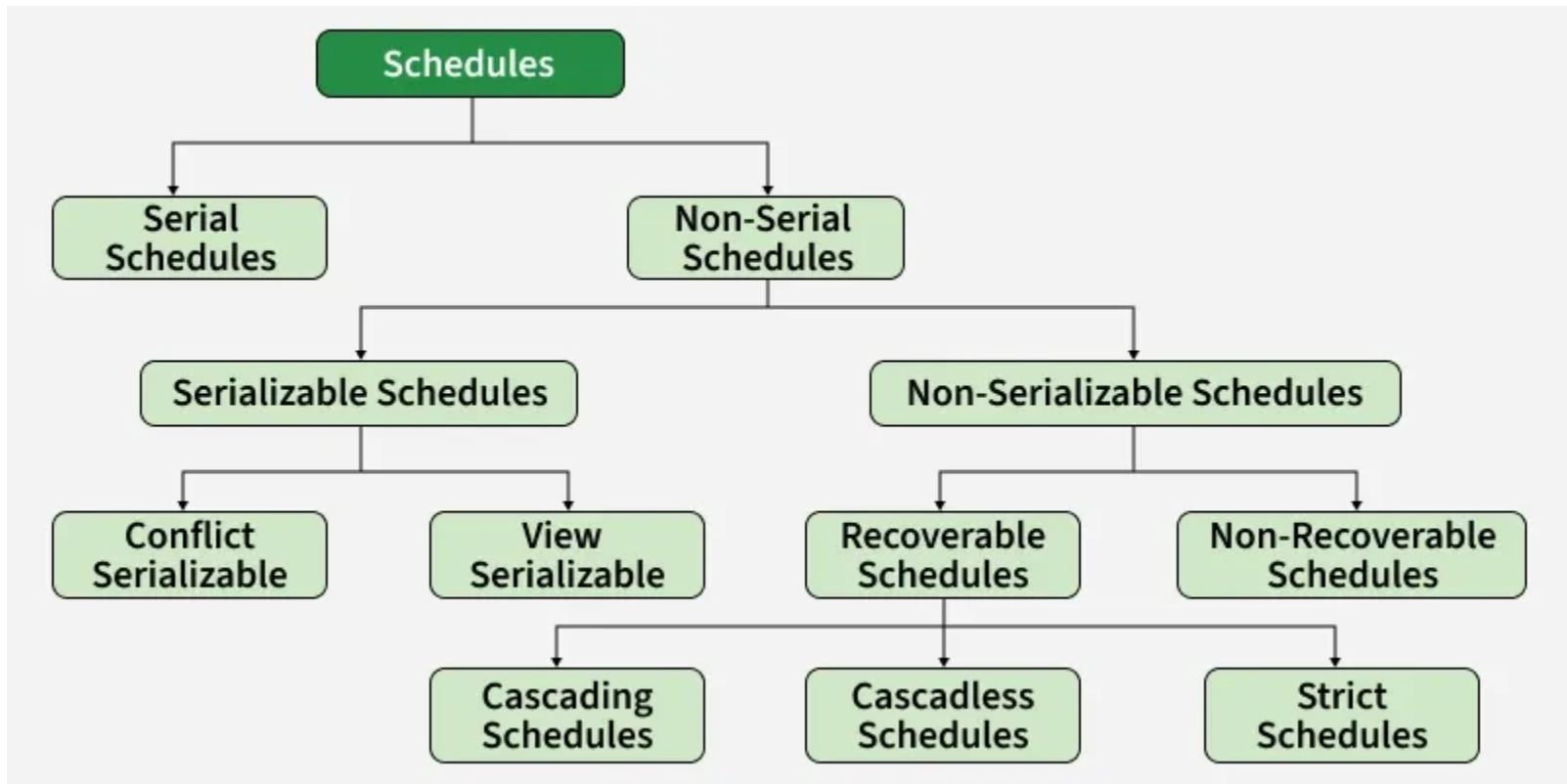
Kill the transaction

Committed – after successful completion.



Scheduling

Scheduling is the process of determining the order in which transactions are executed. When multiple transactions run concurrently, scheduling ensures that operations are executed in a way that prevents conflicts or overlaps between them.



Scheduling

1. Serial Schedule

Schedules in which transactions execute one after another without interleaving, i.e., no transaction starts until a running transaction has ended are called serial schedules.

- Only one transaction runs at a time.
- No concurrency issues.
- Simple and always consistent.

Example: Consider the following schedule involving two transactions T_1 and T_2 .

T_1	T_2
R(A)	
W(A)	
R(B)	
	W(B)
	R(A)
	R(B)

Types of Problems in Concurrency (DBMS)

When multiple transactions execute **concurrently**, several anomalies (problems) can occur.

1. Dirty Read (Uncommitted Read)

Definition

A transaction reads data that is **not yet committed** by another transaction.

Example

- T1 updates X = 200 (not committed)
 - T2 reads X = 200
 - T1 rolls back → X goes back to old value
- T2 used **invalid data** ✗

2. Incorrect Summary (Inconsistent Analysis)

Definition

Occurs when one transaction is **calculating aggregate values** (SUM, AVG) while another transaction is **updating data simultaneously**.

Example

- T1 calculates total balance of accounts
 - T2 updates one account during calculation
- Result is **partially old + partially new data** ✗

Transaction Management

3. Lost Update

Definition

Two transactions update the **same data**, and one update is **overwritten by another**.

Example

- $X = 100$
 - $T1 \rightarrow X = X - 20 \rightarrow$ writes 80
 - $T2 \rightarrow X = X + 10 \rightarrow$ writes 110
- Final value = 110 (T1 update lost) ✗
- ✓ Correct value should be 90

4. Unrepeatable Read

Definition

A transaction reads the same data **twice**, but gets **different values**.

Example

- T1 reads $X = 100$
 - T2 updates $X = 150$ and commits
 - T1 reads X again $\rightarrow$ gets 150
- Values are inconsistent ✗

Transaction Management

5. Phantom Read

Definition

A transaction re-executes a query and finds **new rows inserted or deleted** by another transaction.

Example

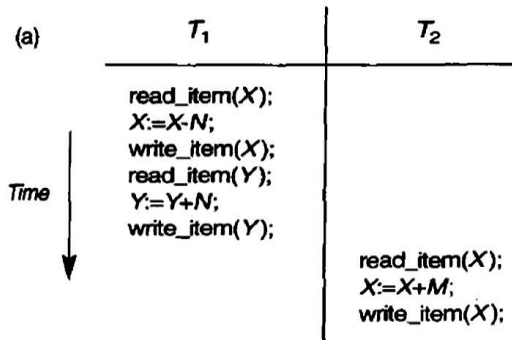
- T1: SELECT * FROM Students WHERE marks > 50
- T2 inserts a new student with marks = 60
- T1 runs query again → sees extra row
“Phantom” row appears ✖

Summary

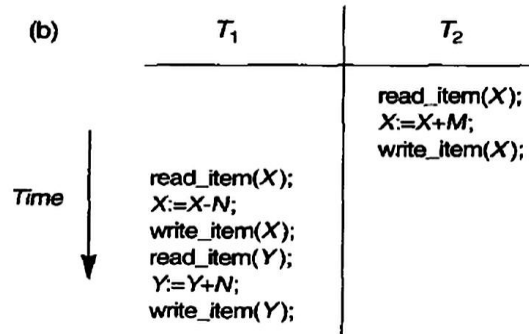
Problem	Issue
Dirty Read	Reads uncommitted data
Incorrect Summary	Aggregation inconsistency
Lost Update	One update overwritten
Unrepeatable Read	Same query, different result
Phantom Read	New rows appear/disappear

Transaction Management

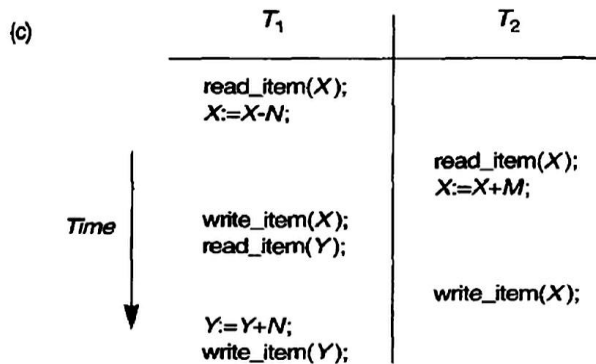
A non-serial schedule in DBMS allows operations from multiple transactions to interleave or execute concurrently, rather than one after another. While they improve CPU throughput and resource utilization, they risk database inconsistency if not properly managed. To ensure consistency, these schedules must be serializable.



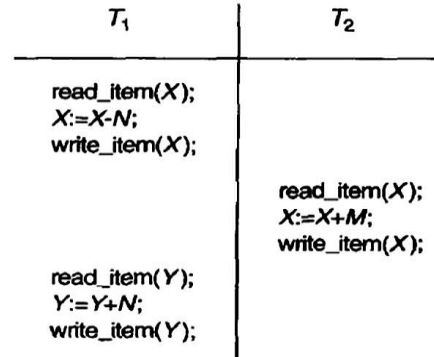
Schedule A



Schedule B



Schedule C



Schedule D

Examples of serial and nonserial schedules involving transactions T_1 and T_2 .

(a) Serial schedule A: T_1 followed by T_2 .

(b) Serial schedule B: T_2 followed by T_1 .

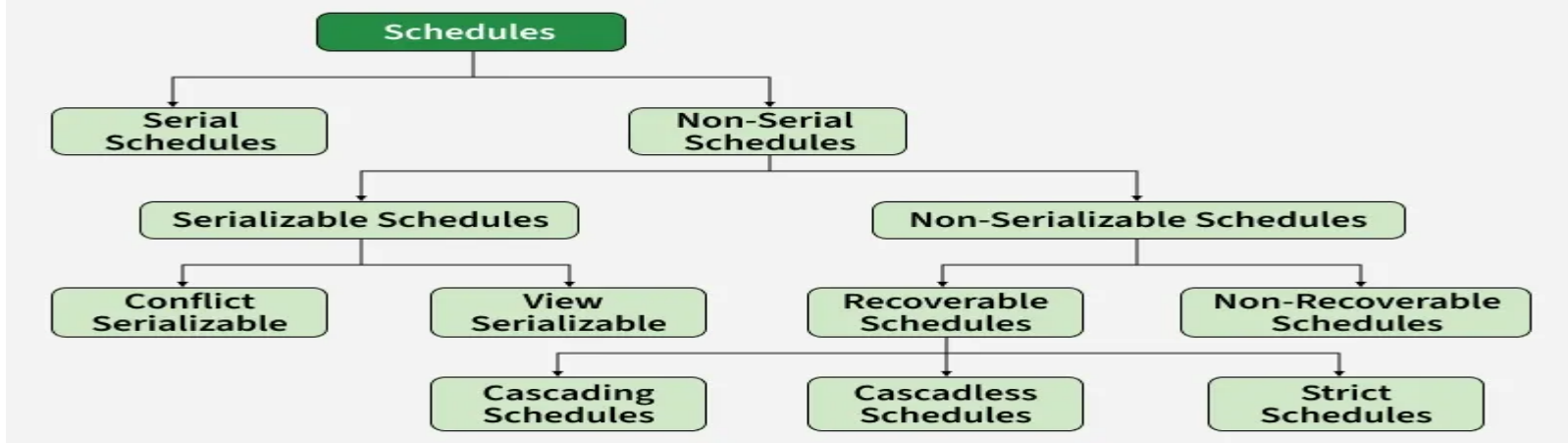
(c) Two nonserial schedules C and D with interleaving of operations.

Transaction Management

Key Aspects of Non-Serial Schedules:

- **Interleaving:** Operations of transactions (T_1, T_2) are mixed (e.g., T_1 reads, then T_2 reads, then T_1 writes).
- **Performance:** Generally more efficient than serial schedules due to parallel processing.
- **Types:**
 - **Serializable:** A non-serial schedule that produces the same final result as a serial execution.
 - **Non-Serializable:** May cause data inconsistency, such as dirty reads or lost updates.
- **Risk:** Can lead to problems like non-recoverable schedules if not properly ordered, where one transaction commits before another dependent transaction does.

Transaction Management



Non-Serial Schedule

Transactions execute in an interleaved manner in Non-Serial Schedule.

- Allows concurrency.
- Must be checked for correctness.
- May or may not be serializable.

Non-serial schedules are of two types:

- Serializable Scheduling (Concurrency Control)
- A non-serial schedule that behaves like a serial schedule.

Why Serializable Schedules?

- Maintain consistency.
- Avoid anomalies.
- Improve performance by allowing concurrency.

Serializable Scheduling

There are two types of serializable schedules:

- i. **Conflict Serializable**
- ii. **View Serializable**

Conflict Serializable

Conflict Serializability ensures that a concurrent schedule produces the same result as some serial execution by reordering non-conflicting operations. It maintains data consistency and is stricter than View Serializability, which allows more flexibility but still preserves correctness.

•**Non-conflicting operations:** Two operations are considered non-conflicting if **they operate on separate data items, or if they involve the same data item but both are read operations.**

Serializable Scheduling

Conflict Serializability ensures that a concurrent schedule produces the same result as some serial execution by reordering non-conflicting operations. It maintains data consistency and is stricter than View Serializability, which allows more flexibility but still preserves correctness.

- Non-conflicting operations:** Two operations are considered non-conflicting if they operate on separate data items, or if they involve the same data item but both are read operations.

Serializable Scheduling

Conflicting Operations

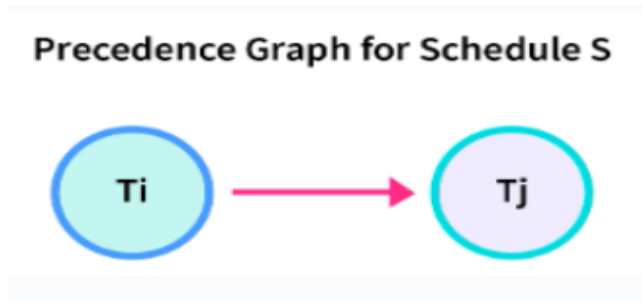
Two operations are said to be conflicting if all conditions are satisfied:

- They belong to different transactions
- They operate on the same data item
- Atleast one of them is a write operation

Operation by T_i	Operation by T_j	Conflict Status
read(Q)	read(Q)	No Conflict
read(Q)	write(Q)	Conflict
write(Q)	read(Q)	Conflict
write(Q)	write(Q)	Conflict

Serializable Scheduling

Testing of Serializability To test the serializability of a schedule, we can use Serialization Graph or Precedence Graph. A serialization Graph is nothing but a Directed Graph of the entire transactions of a schedule. It can be defined as a Graph $G(V, E)$ consisting of a set of directed-edges $E = \{E_1, E_2, E_3, \dots, E_n\}$ and a set of vertices $V = \{V_1, V_2, V_3, \dots, V_n\}$. The set of edges contains one of the two operations - READ, WRITE performed by a certain transaction.

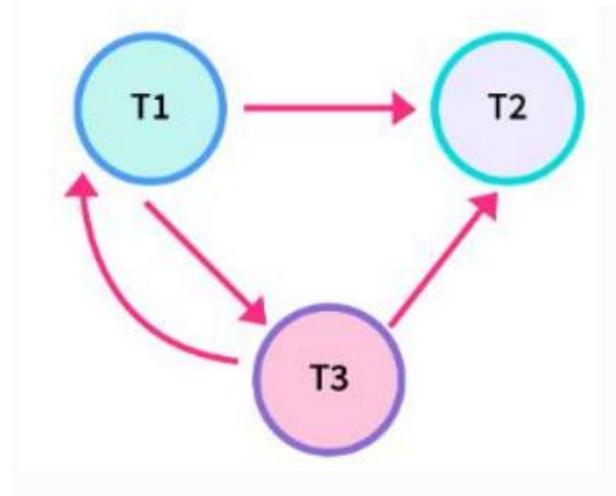


$T_i \rightarrow T_j$, means Transaction- T_i is either performing read or write before the transaction- T_j .

Transaction Management

Let's take an example of schedule "S" having three transactions t1, t2, and t3 working simultaneously

t1	t2	t3
R(x)		
		R(z)
		W(z)
	R(y)	
R(y)		
	W(y)	
		W(x)
	W(z)	
W(x)		

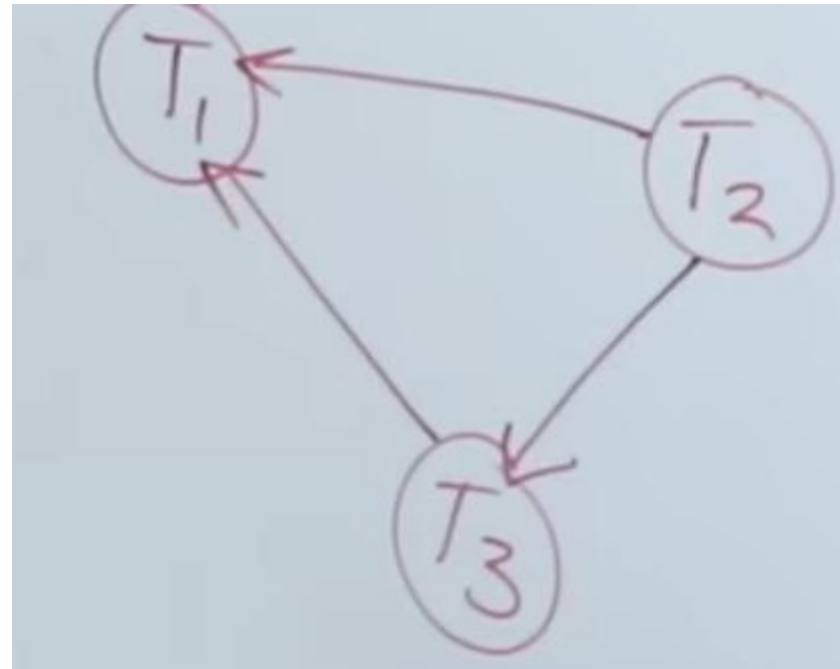


Non-serializable schedule. R(x) of T1 conflicts with W(x) of T3, so there is a directed edge from T1 to T3. R(y) of T1 conflicts with W(y) of T2, so there is a directed edge from T1 to T2. W(x) of T3 conflicts with W(x) of T1, so there is a directed edge from T3 to T. Similarly, we will make edges for every conflicting pair. Now, as the cycle is formed, the transactions cannot be serializable

Serializable Scheduling

What is Conflict Serializability in DBMS? Conflict Serializability checks if a non-serial schedule is conflict serializable or not. A non-serial schedule is conflict serializable if it can convert into a serial schedule by swapping its nonconflicting operations.

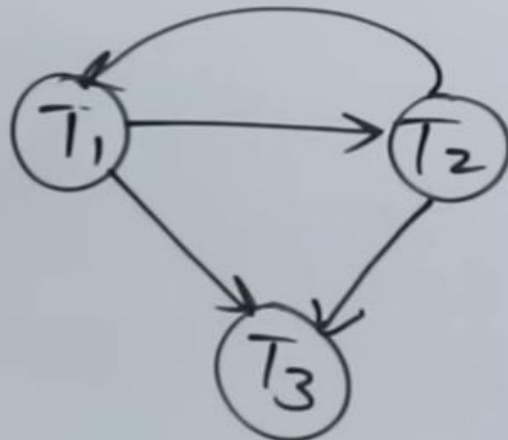
T_1	T_2	T_3
<u>R(x)</u>		
		R(y)
		R(x)
	R(y)	
	R(z)	
	w(z)	w(y)
R(z)		
w(x)		
w(z)		



Serializable Scheduling

T_1	T_2	T_3
$R(A)$		
	$W(A)$	
$W(A)$		
		$W(A)$

T_1	T_2	T_3
$R(A)$		
$W(A)$		
	$W(A)$	
		$W(A)$



view serializable

Serializable Scheduling

A view serializable schedule in DBMS is a non-serial schedule that is [view equivalent](#) to a serial schedule, ensuring data consistency equivalent to running transactions one after another. It maintains the same initial reads, read-write dependencies, and final writes as a serial schedule. It is more comprehensive than conflict serializability, particularly covering cases with blind writes.

Key Conditions for View Equivalence (S1 to S2)

For a schedule (S1) to be view serializable, it must satisfy these conditions with a serial schedule (S2):

- **Initial Read:** If a transaction T_i reads the initial value of data A in S1, it must also read the initial value of A in S2.
- **Updated Read (Read-Write Mapping):** If T_i reads a value of A written by T_j in S1, it must read the value of A written by T_j in S2.
- **Final Write:** The transaction that performs the final write on a data item A in S1 must also perform the final write on A in S2.

Example

Consider a non-serial schedule S with T_1 and T_2 where T_1 updates A and T_2 updates A later. If S is rearranged into a serial order ($T_1 \rightarrow T_2$ or $T_2 \rightarrow T_1$) and all read/write dependencies match, S is view serializable.

Serializable Scheduling

S1	T ₁	T ₂	S2	T ₁	T ₂
	R(A)	W(A)		R(A)	W(A)

Initial Read

ALSO.

S1	T ₁	T ₂	T ₃	S2	T ₁	T ₂	T ₃
	W(A)	W(A)	R(A)		W(A)	W(A)	R(A)

Update Read

T ₁	T ₂	T ₃	T ₁	T ₂	T ₃
W(A)	R(A)	W(A)	W(A)	R(A)	W(A)

Final Write

Serializable Scheduling

Non-Serial

S1

T1	T2
R(A)	
W(A)	
	R(A)
	W(A)
R(B)	
W(B)	
	R(B)
	W(B)

Serial

S2

T1	T2
R(A)	
W(A)	
R(A)	
W(A)	
	R(B)
	W(B)
	R(B)
	W(B)

Let's look at the three view serializability conditions:

1.Initial Read

First, we will see that transaction T1 in schedule S1 reads data item A first.

Transaction T1 in S2 also reads data item A first. Then look for B. Transaction T1 in schedule S1 reads data item B first. T1 also performs the first read operation on B in S2. We checked both data items A and B, and the initial read condition is met in S1 and S2.

Serializable Scheduling

2. Update Read

The second method is about update read operation. Transaction T2 in S1 reads the value of A written by T1. In the same transaction, T2 reads the A after it is written by T1 in S2. Transaction T2 in S1 reads the value of B written by T1. The same transaction T2 reads the value of B after it has been updated by T1 in S2. Both schedules also satisfy the update read condition.

3. Final Write

In schedule S1, the final write operation on A is done by transaction T2. Transaction T2 also performs the final write on A in S2. Let's look for B. Transaction T2 performs the final write operation on B in schedule S1. T2 completes the final write on B in schedule S2. We checked both data items A and B, and the final write condition is met in S1 and S2.

Non-Serializable Scheduling

Non-Serializable Scheduling

Schedules that do not preserve serial equivalence and may lead to inconsistencies if not handled carefully.

They are classified into the following types:

i. **Recoverable Schedule** ii. **Cascading Schedule** iii. **Cascadeless Schedule**

i. **Recoverable Schedule**

Recoverable schedules are those in which a transaction commits only after all transactions whose values it has read have also committed.

Key Point: If T_2 reads a value written by T_1 , then T_2 must commit after T_1 .

Example: Consider the following schedule involving two transactions T_1 and T_2 .

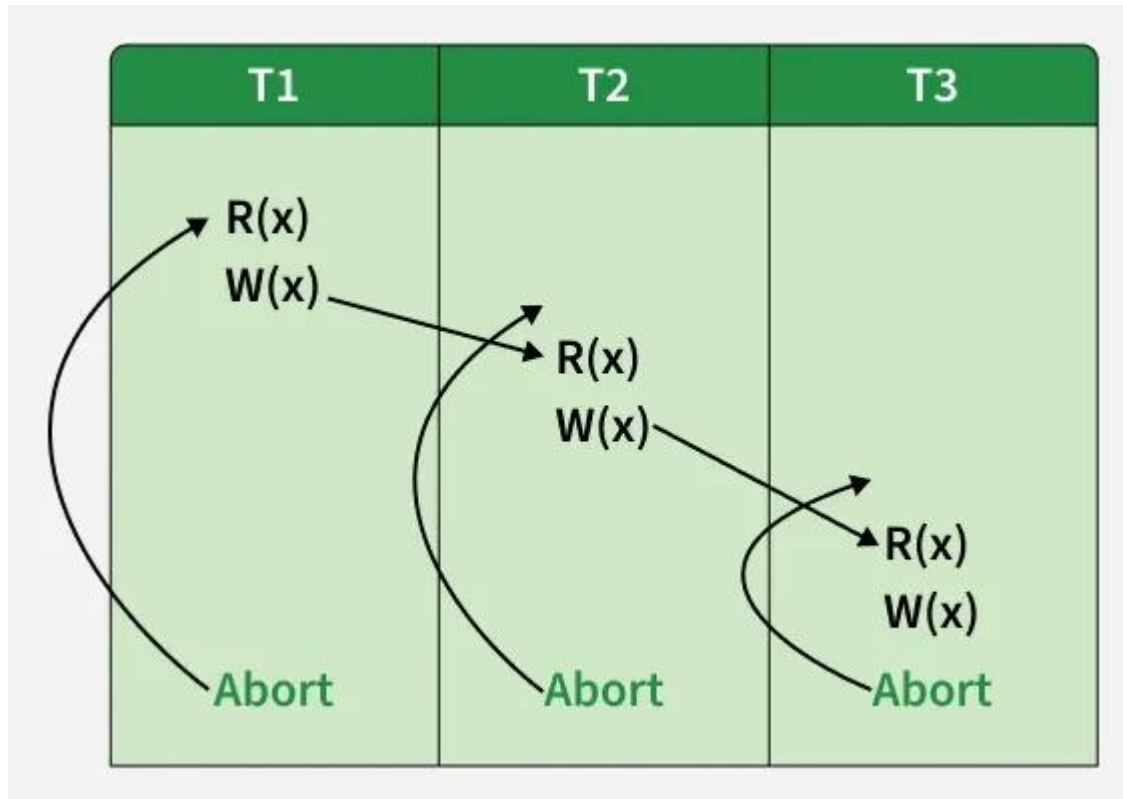
T_1	T_2
R(A)	
W(A)	
	W(A)
	R(A)
Commit	
	Commit

This is a recoverable schedule since T_1 commits before T_2 , that makes the value read by T_2 correct. There can be three types of recoverable schedule:

Recoverable Schedule

ii Cascading Schedule: A schedule where:

- Failure of one transaction forces all dependent transactions to abort.
- Happens when transactions read uncommitted data.



Recoverable Schedule

iii. Cascadeless Schedule: A schedule where:

- A transaction only reads committed data.
- Prevents cascading aborts.
- Key Point: T2 can read the value written by T1 only after T1 commits.

Example: Consider the following schedule involving two transactions T_1 and T_2 .

T_1	T_2
R(A)	
W(A)	
	W(A)
Commit	
	R(A)
	Commit

This schedule is cascadeless. Since the updated value of **A** is read by T_2 only after the updating transaction i.e. T_1 commits.

Recoverable Schedule

iv Strict Schedule: It's a stronger form of cascadeless schedule that follows the following rules:

- A transaction cannot read or write a data item written by another transaction until that transaction commits or aborts.
- Prevents dirty reads and dirty writes.

Example: Consider the following schedule involving two transactions T_1 and T_2 .

T_1	T_2
R(A)	
	R(A)
W(A)	
Commit	
	W(A)
	R(A)
	Commit

This is a strict schedule since T_2 reads and writes A which is written by T_1 only after the commit of T_1 .

Non-Recoverable Schedule

Non-Recoverable Schedule: A schedule where:

- A transaction commits after reading **uncommitted data** from another transaction.
- If the writer transaction later aborts, the schedule becomes invalid.
- Key Point: These schedules must be avoided since they lead to inconsistencies.

Example: Consider the following schedule involving two transactions T1 and T2.

T ₁	T ₂
R(A)	
W(A)	
	W(A)
	R(A)
	Commit
abort	

T₂ read the value of A written by T₁, and committed. T₁ later aborted, therefore the value read by T₂ is wrong, but since T₂ committed, this schedule is non-recoverable.

Concurrent Execution of Transactions

Concurrent execution allows **multiple transactions to be processed simultaneously** instead of one after another. This improves:

- **Throughput (more work done in less time)**
- **Resource utilization (CPU, memory, I/O used efficiently)**
- **Response time for users**

Problems with Concurrent Execution

Without proper control, concurrency can cause **inconsistencies**:

1. Lost Update Problem

1. Two transactions update the same data; one overwrites the other.

2. Dirty Read (Uncommitted Read)

1. A transaction reads data modified by another transaction that hasn't committed.

3. Non-repeatable Read

1. Same query gives different results within the same transaction.

4. Phantom Read

1. New rows appear/disappear during transaction execution.

Lock-Based Concurrency Control

It is a mechanism where a **transaction must acquire a lock on data** before accessing it. This ensures that multiple transactions **do not interfere** with each other.

A **lock** is a mechanism used by the DBMS to **control access to data items** when multiple transactions execute at the same time.

In simple terms: **Lock = Permission to access data**

Why Locks are Needed?

When transactions run concurrently, problems like:

- Lost updates
- Dirty reads
- Inconsistent data

Locks help **maintain data consistency and isolation**

Types of Lock

1. Shared Lock (S-Lock) – Read Lock: Shared Lock is also known as **Read-only lock**. As the name suggests it can be shared between transactions because while holding this lock the transaction does not have the permission to update data on the data item. S-lock is requested using lock-S instruction.

Example:

T1 reads A → S-lock(A)

T2 reads A → Allowed

2. Exclusive Lock (X-Lock) – Write Lock : Data item can be both read as well as written. This is Exclusive and cannot be held simultaneously on the same data item. X-lock is requested using lock-X instruction.

Example:

T1 writes A → X-lock(A)

T2 tries to read/write A → Blocked

Transaction Management

Read Lock (or) Shared Lock(S)

- If a Transaction has a Read lock on a data item, it can read the item but not update it.
- If a transaction has a Read lock on the data item, other transaction can obtain Read Lock on the data item but no Write Locks.
- So, the Read Lock is also called a Shared Lock.

Write Lock (or) Exclusive Lock (X)

- If a transaction has a write Lock on a data item, it can both read and update the data item.
- If a transaction has a write Lock on the data item, then other transactions cannot obtain either a Read lock or write lock on the data item.
- So, the Write Lock is also known as Exclusive Lock.

Transaction Management

Lock Compatibility Matrix

- **Shared Locks (S):** Multiple transactions can hold shared locks on the same data item simultaneously.
- **Exclusive Lock (X):** If a transaction holds an exclusive lock on a data item, no other transaction can hold any type of lock on that item.

	S	X
S	✓	X
X	X	X

Transaction Management

Types of Lock-Based Protocols

1. Simplistic Lock Protocol: It is the simplest method for locking data during a transaction. Simple lock-based protocols **enable all transactions to obtain a lock on the data before inserting, deleting, or updating it.** It will unlock the data item once the transaction is completed.

Example: Consider a database with a single data item $X = 10$.

Transactions:

T1: Wants to read and update X.

T2: Wants to read X.

Steps:

T1 requests an exclusive lock on X to update its value. The lock is granted.

T1 reads $X = 10$ and updates it to $X = 20$.

T2 requests a shared lock on X to read its value. Since T1 is holding an exclusive lock, T2 must wait.

T1 completes its operation and releases the lock.

T2 now gets the shared lock and reads the updated value $X = 20$.

This example shows how simplistic lock protocols handle concurrency but do not prevent problems like deadlocks or limits concurrency.

Transaction Management



2. Pre-Claiming Lock Protocol

The Pre-Claiming Lock Protocol avoids deadlocks by requiring a transaction to request all needed locks before it starts. **It runs only if all locks are granted**; otherwise, it waits or rolls back.

Example: Consider two transactions T1 and T2 and two data items, X and Y:

Transaction T1 declares that it needs:

- A write lock on X.
- A read lock on Y.

Since both locks are available, the system grants them. T1 starts execution: It updates X. It reads the value of Y.

While T1 is executing, Transaction T2 declares that it needs: However, since T1 already holds a write lock on X, T2's request is denied. T2 must wait until T1 completes its operations and releases the locks. A read lock on X

Once T1 finishes, it releases the locks on X and Y. The system now grants the read lock on X to T2, allowing it to proceed.

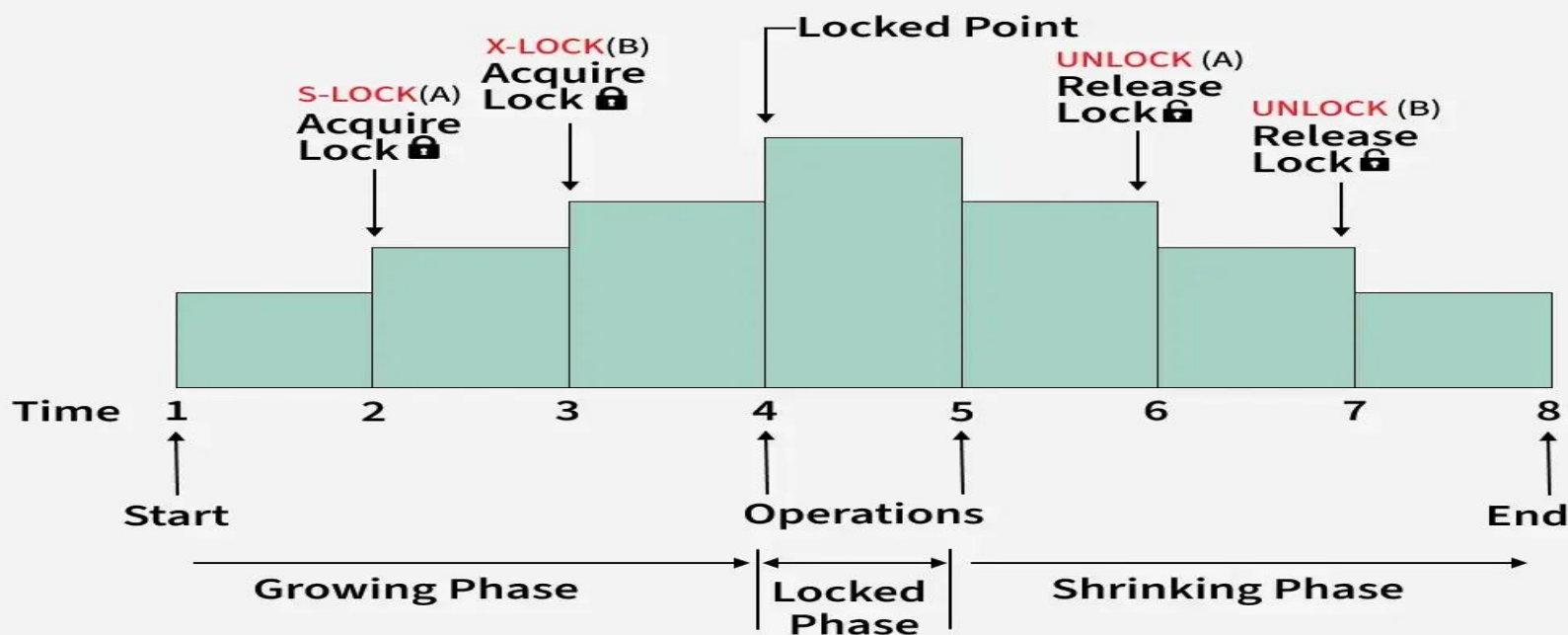
This method is simple but may lead to inefficiency in systems with a high number of transactions.

Transaction Management

3. Two-phase locking (2PL)

A transaction is said to follow the [Two-Phase Locking](#) protocol if Locking and Unlocking can be done in two phases :

- **Growing Phase:** New locks on data items may be acquired but none can be released.
- **Shrinking Phase:** Existing locks may be released but no new locks can be acquired.



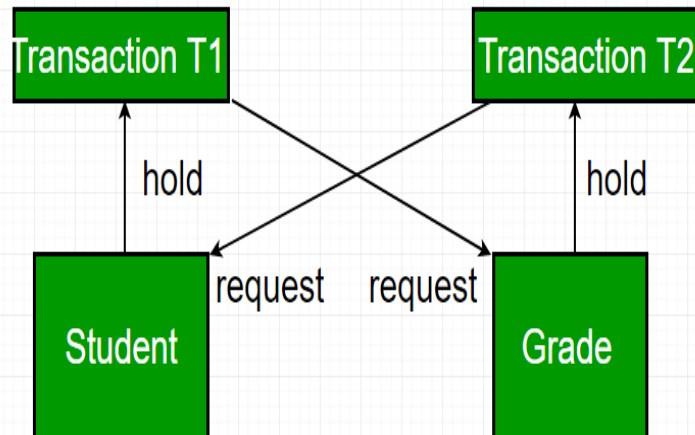
Transaction Management

4. Strict Two-Phase Locking Protocol

Strict Two-Phase Locking requires that in addition to the 2-PL all Exclusive(X) locks held by the transaction be released until after the Transaction Commits.

Deadlock

A deadlock occurs in a multi-user database environment when two or more transactions block each other indefinitely by each holding a resource the other needs. This results in a cycle of dependencies (circular wait) where no transaction can proceed.



T1 locks Resource "Student" and needs Resource "Grade"
T2 locks Resource "Grade" and needs Resource "Student"
T1 waits for T2, T2 waits for T1, hence resulting in a deadlock

Transaction Management

Necessary Conditions for Deadlock

(All must occur together)

1. **Mutual Exclusion** :Resource (data item) is used by one transaction at a time
2. **Hold and Wait** :Transaction holds one lock and waits for another
3. **No Preemption** :Locks cannot be forcibly taken away
4. **Circular Wait** :Transactions form a circular chain

Example: $T1 \rightarrow T2 \rightarrow T3 \rightarrow T1$

Deadlock Detection

Wait-For Graph (WFG)

- Nodes $\rightarrow$ Transactions
 - Edge $T_i \rightarrow T_j \rightarrow T_i$ is waiting for T_j
- If graph contains a **cycle** $\rightarrow$ Deadlock exists

Example

- $T1 \rightarrow T2$
- $T2 \rightarrow T3$
- $T3 \rightarrow T1$

Cycle présent $\rightarrow$ Deadlock

Deadlock Prevention or avoidance

A deadlock can be prevented if the resources are allocated in such a way that a deadlock never occurs. The DBMS analyzes the operations whether they can create a deadlock situation or not, If they do, that transaction is never allowed to be executed.

1. Wait-Die Scheme (Non-preemptive)

- Older transactions are allowed to wait.
- Younger transactions are killed (aborted and restarted) if they request a resource held by an older one

For example:

- Consider two transaction- $T1 = 10$ and $T2 = 20$
- If $T1$ (older) wants a resource held by $T2 \rightarrow T1$ waits
- If $T2$ (younger) wants a resource held by $T1 \rightarrow T2$ dies and restarts

Prevents deadlock by not allowing a younger transaction to wait and form a wait cycle

2. Wound-Wait Scheme (Preemptive)

- Older transactions are aggressive (preemptive) and can force younger ones to abort.
- Younger transactions must wait if they want a resource held by an older one.

For example:

- Consider two transaction- $T1 = 10$ and $T2 = 20$
- If $T1$ (older) wants a resource held by $T2 \rightarrow T2$ is killed, $T1$ proceeds.
- If $T2$ (younger) wants a resource held by $T1 \rightarrow T2$ waits

Prevents deadlock by not allowing younger transactions to block older ones.

Transaction Management

Wait - Die	Wound -Wait
It is based on a non-preemptive technique.	It is based on a preemptive technique.
In this, older transactions must wait for the younger one to release its data items.	In this, older transactions never wait for younger transactions.
The number of aborts and rollbacks is higher in these techniques.	In this, the number of aborts and rollback is lesser.

Applications Affected by Deadlock

- 1.Delayed Transactions:** blocked indefinitely
- 2.Lost Transactions:** may get aborted or rolled back
- 3.Poor User Experience:** slow response or system freeze

Transaction Management

Recovery from Deadlock

(a) Selection of a Victim :

Given a set of deadlocked transactions, one must determine which transaction has to be rolled back to break the deadlock. Then, that transaction should be rolled back which occurs at **minimum cost**. Many other factors also play a role in deciding which transactions need to be rolled back

(b) Rollback :

Once it is decided that a particular transaction must be rolled back, then determine how far this transaction should be rolled back.

The simplest solution is '**Total Rollback**'. That is, we abort the transaction and then restart it

Sometimes, it is possible to do '**partial rollback**', which retains the consistency of the database. 'Partial Rollback' requires the system to maintain additional information about the state of all the running transactions.

Recovery from Deadlock

(c)Starvation:

In a system where the selection of a transaction is based primarily on cost factors, then it is possible to pick the same transaction to be rolled back, known as '**victim**'. As a result, the transaction never completes its designated task, thus there is a '**starvation**'. So, here one should decide that a selected transaction be rolled back for a finite number of times only. Then 'starvation' does not occur.

Concurrency Control Techniques

Concurrency control is a fundamental concept in database systems that ensures correct execution of simultaneous transactions without violating data integrity.

Below are some characteristics of concurrency control:

- It enforces isolation among transactions.
- It preserve database consistency through consistency preserving execution of transactions.
- It resolve read-write and write-read conflicts.

Concurrency control techniques

1. *Two-phase locking Protocol*
2. *Time stamp ordering Protocol*
3. *Multi version concurrency control*
4. *Validation concurrency control*

Transaction Management

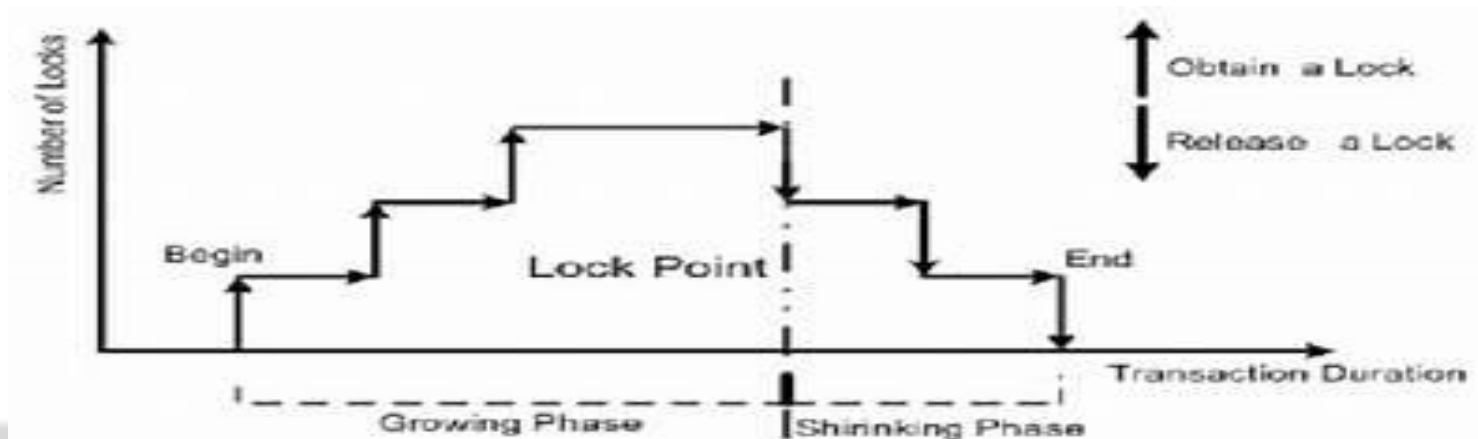
1. Two-Phase Locking Protocol

Locking is an operation which secures: permission to read, OR permission to write a data item. Two phase locking is a process used to gain ownership of shared resources without creating the possibility of deadlock. The 3 activities taking place in the two phase update algorithm are:

1.Lock Acquisition

2.Modification of Data

3.Release Lock



Transaction Management

Lock Point

The lock point in a transaction is the moment when the transaction finishes acquiring all the locks it needs. After this point, no new locks can be added, and the transaction starts releasing locks. It's a key step in the Two-Phase Locking Protocol to ensure the rules of growing and shrinking phases are followed.

Simple Example

Consider **two transactions**:

- T1**: Read and update X
- T2**: Read X

Execution with 2PL

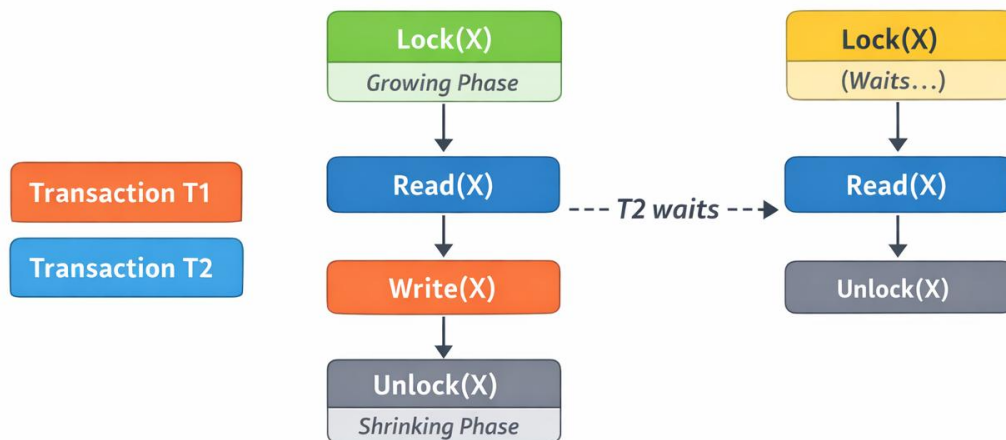
Transaction T1:

1. Lock(X) → (Growing phase)
2. Read(X)
3. Write(X)
4. Unlock(X) → (Shrinking phase starts)

Transaction T2:

1. Lock(X) → waits until T1 releases
2. Read(X)
3. Unlock(X)

Two-Phase Locking (2PL) Example



Transaction Management

Two-phase locking prevents deadlocks by ensuring a process releases all held locks if it can't acquire all needed resources without waiting. This avoids situations where processes wait on each other, preventing deadlock.

A transaction in the Two Phase Locking Protocol can assume one of the 2 phases:

- Growing Phase:** In this phase a transaction can only acquire locks but cannot release any lock. The point when a transaction acquires all the locks it needs is called the Lock Point.
- Shrinking Phase:** In this phase a transaction can only release locks but cannot acquire any.

Concurrency control without locking:

Timestamp Ordering Protocol

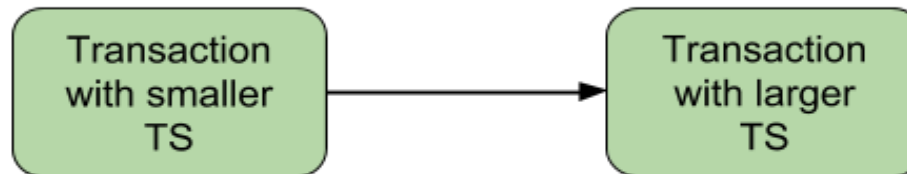
The Timestamp Ordering Protocol enforces that older transactions (with smaller timestamps) are given higher priority. This prevents conflicts and ensures the execution is serializable and deadlock-free.

Transaction Management

For example:

- If Transaction T1 enters the system first, it gets a timestamp $TS(T1) = 007$ (assumption).
- If Transaction T2 enters after T1, it gets a timestamp $TS(T2) = 009$ (assumption).

This means T1 is "older" than T2 and T1 should execute before T2 to maintain consistency.



Validation Concurrency Control

The optimistic approach assumes that conflicts between transactions are rare. So, it allows transactions to execute freely without using locks or timestamps. Each transaction goes through three phases:

- 1. Read Phase:** The transaction reads data and performs all operations using a private copy. Any updates are stored in a temporary area, not visible to other transactions.
- 2. Validation Phase:** Before committing, the transaction is checked to ensure that applying its changes won't violate database consistency.
 - If the check **passes**, it proceeds to the next phase.
 - If the check **fails**, the transaction is rolled back and restarted.
- 3. Write Phase:** The validated changes are now written permanently to the database.

Multiversion Concurrency Control (MVCC)

is a technique used in databases to **allow many users to read and update data at the same time without blocking each other.**

Instead of overwriting a record when it is updated, the database creates a new version of that record. This means:

- Readers can continue viewing the **old version** safely.
- Writers update a **new version** without stopping others.
- No read/write blocking happens.

MVCC helps prevent conflicts and reduces the need for strict locking, which keeps the database fast and responsive even when many users are working at once.

Crash Recovery: Aries, Recovering from a System Crash

Algorithm for Recovery and Isolation Exploiting Semantics (ARIES)

ARIES is a robust recovery algorithm based on the Write-Ahead Logging (WAL) protocol, widely used in modern DBMS to ensure durability and consistency after system failures.

Types of Log Records in ARIES

Every update operation creates a **log record**, which can be of the following types:

- **Undo-only Log Record:** Stores only the **before image** (original data). Used to undo changes.
- **Redo-only Log Record:** Stores only the **after image** (new data). Used to redo changes.
- **Undo-Redo Log Record:** Stores both **before and after** images. Used for both undo and redo operations.

Transaction Management

Log Records

Each action is stored like:

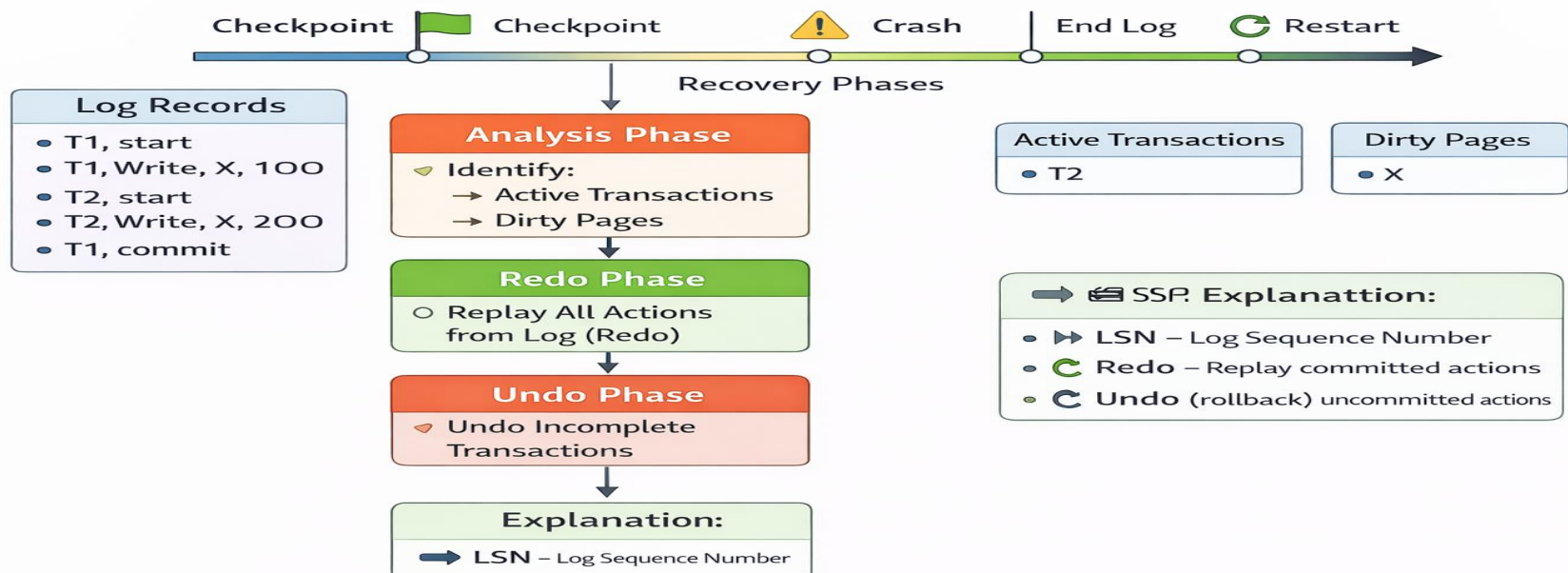
• <T1, Write, X, old value, new value>

Checkpoints

Save system state to reduce recovery time, At checkpoint, DBMS stores:

1. Active transactions list
2. Dirty pages (modified but not written to disk)

ARIES Recovery Process



Transaction Management

Transaction Management